



به نام خدا

دانشگاه تهران



دانشکده مهندسی برق و کامپیوتر

درس شبکه‌های عصبی و یادگیری عمیق تمرین امتیازی

نام و نام خانوادگی	سید مانی میرشعبانی	پرسش ۱
شماره دانشجویی	810801080	
نام و نام خانوادگی	سید مانی میرشعبانی	پرسش ۲
شماره دانشجویی	810801080	

فهرست

1. قوانین.....
3. پرسش 1. تحلیل عملکرد شبکه های عصبی تحت حملات متخاصم.....
3. 1-1. آموزش ResNet روی تصاویر نویزی.....
3. 1-1-1. بارگذاری مجموعه داده.....
3. 1-1-2. افزودن نویز.....
4. 1-1-3. آموزش مدل ResNet.....
5. 1-1-4. نمودارهای دقت و خطا.....
6. 1-1-5. مقایسه عملکرد مدل ها.....
8. 1-2. یادگیری انتقالی روی مجموعه داده Flowers-102 با ViT.....
8. 1-2-0. بارگذاری و پیش پردازش مجموعه داده.....
8. 1-2-1. بارگذاری مدل از پیش آموزش دیده.....
9. 1-2-2. Fine Tuning روی Flowers-102.....
9. 1-2-2. بارگذاری و آموزش ViT از ابتدا.....
12. 1-3. حملات متخاصم و دفاع.....
13. 1-3-1. پیاده سازی FGSM و PGD.....
15. 1-3-2. تولید تصاویر متخاصم و ارزیابی مدل ها.....
16. 1-3-3. Adversarial Training.....
17. 1-4. سؤالات تئوری.....
17. 1-4-1. با وجود اینکه مدل ResNet دارای میلیون ها پارامتر است، چرا در بسیاری از موارد با داده های نویزی همچنان عملکرد مناسبی دارد؟
19. 2-4-1. مدل های ViT (Transformer Vision) در صورت استفاده از وزن های پیش آموزش دیده بهتر از حالت بدون pretrain عمل میکنند. این اختلاف را با استفاده از مفاهیم مینیمم تیز (sharp) و تخت (flat) تحلیل کنید.....
20. 3-4-1. تحلیل آموزش پادمانده (adversarial training) به عنوان شکلی از افزایش داده (data augmentation).....
20. 3-4-1. adversarial training را به عنوان شکلی از data augmentation تحلیل کنید.....
21. 4-4-1. با تحلیل ساختار ResNet و ViT و استفاده از مفاهیم dropout و ensemble ، تفاوت این دو مدل در برابر حملات متخاصم را توجیه کنید .

شکل‌ها

- شکل 1: نمونه یک عکس بدون نویز و یک عکس با نویز (عکس‌ها از دو کلاس متفاوت هستند، چرا که بعد از درست کردن data loader ها تصاویر بر (shuffle) میخورند به همین جهت عکس نویز دار حاصل اضافه کردن نویز به عکس نمایش داده شده نیست!).....3
- شکل 2: نتیجه آموزش مدل‌ها بدون L2, Dropout و.....4
- شکل 3: نتیجه مدل‌ها بعد از اضافه کردن L2 Regularization.....4
- شکل 4: نتیجه مدل‌ها بعد از اضافه کردن Scheduler.....5
- شکل 5: نمودار عملکرد مدل‌های ResNet.....6
- شکل 6: جدول عملکرد مدل‌های ResNet.....7
- شکل 7: نمونه تصاویر Flowers-102 بعد از لود.....8
- شکل 8: تعداد پارامترهای مدل از پیش آموزش دیده ViT.....9
- شکل 9: روند آموزش مدل از پیش آموزش دیده ViT.....9
- شکل 10: روند یادگیری مدل بدون آموزش اولیه.....10
- شکل 11: نمودار خطا و دقت مدل‌های ViT در طول آموزش.....11
- شکل 12: فرمول محاسبه FGSM.....12
- شکل 13: تابع fgsm.....14
- شکل 14: تابع pgd.....15
- شکل 15: نتایج مدل‌های بعد از حمله.....15
- شکل 16: عملکرد مدل‌ها بعد از Adversarial training.....16
- شکل 17: ساختار ResNet.....17
- شکل 18: نتایج ResNet در برابر نویز.....19
- شکل 19: ساختار ViT.....22

جدول ها

قبل از پاسخ دادن به پرسش‌ها، موارد زیر را با دقت مطالعه نمایید:

- از پاسخ‌های خود یک گزارش در قالبی که در صفحه‌ی درس در سامانه‌ی Elearn با نام [REPORTS_TEMPLATE.docx](#) قرار داده شده تهیه نمایید.
- پیشنهاد می‌شود تمرین‌ها را در قالب گروه‌های دو نفره انجام دهید. (بیش از دو نفر مجاز نیست و تحویل تک نفره نیز نمره‌ی اضافی ندارد) توجه نمایید الزامی در یکسان ماندن اعضای گروه تا انتهای ترم وجود ندارد. (یعنی، می‌توانید تمرین اول را با شخص A و تمرین دوم را با شخص B و ... انجام دهید)
- **کیفیت گزارش شما در فرآیند تصحیح از اهمیت ویژه‌ای برخوردار است؛** بنابراین، لطفاً تمامی نکات و فرضیه‌ی را که در پیاده‌سازیها و محاسبات خود در نظر می‌گیرید در گزارش ذکر کنید.
- در گزارش خود مطابق با آنچه در قالب نمونه قرار داده شده، برای شکل‌ها زیرنویس و برای جدول‌ها بالانویس در نظر بگیرید.
- الزامی به ارائه توضیح جزئیات کد در گزارش نیست، اما باید نتایج بدست آمده از آن را گزارش و تحلیل کنید.
- **تحلیل نتایج الزامی می‌باشد، حتی اگر در صورت پرسش اشاره‌ای به آن نشده باشد.**
- **دستیاران آموزشی ملزم به اجرا کردن کدهای شما نیستند؛** بنابراین، هرگونه نتیجه و یا تحلیلی که در صورت پرسش از شما خواسته شده را به طور واضح و کامل در گزارش بیاورید. در صورت عدم رعایت این مورد، بدیهی است که از نمره تمرین کسر میشود.
- **کدها حتماً باید در قالب نوت‌بوک با پسوند .ipynb تهیه شوند، در پایان کار، تمامی کد اجرا شود و خروجی هر سلول حتماً در این فایل ارسالی شما ذخیره شده باشد.** بنابراین برای مثال اگر خروجی سلولی یک نمودار است که در گزارش آورده‌اید، این نمودار باید هم در گزارش هم در نوت‌بوک کدها وجود داشته باشد.
- **در صورت مشاهده‌ی تقلب امتیاز تمامی افراد شرکت‌کننده در آن، 100- لحاظ میشود.**
- تنها زبان برنامه نویسی مجاز **Python** است.
- **استفاده از کدهای آماده برای تمرینها به هیچ وجه مجاز نیست.**
- **در صورتی که دو گروه از یک منبع مشترک استفاده کنند و کدهای مشابه تحویل دهند، تقلب محسوب می‌شود.**

- نحوه محاسبه تاخیر به این شکل است: پس از پایان رسیدن مهلت ارسال گزارش، حداکثر تا یک هفته امکان ارسال با تاخیر وجود دارد، پس از این یک هفته نمره آن تکلیف برای شما صفر خواهد شد.

0 سه روز اول: بدون جریمه

0 روز چهارم: ۵ درصد

0 روز پنجم: ۱۰ درصد

0 روز ششم: ۱۵ درصد

0 روز هفتم: ۲۰ درصد

- حداکثر نمره‌ای که برای هر سوال می‌توان اخذ کرد ۱۰۰ بوده و اگر مجموع بارم یک **سوال** بیشتر از ۱۰۰ باشد، در صورت اخذ نمره بیشتر از ۱۰۰، اعمال نخواهد شد.

0 برای مثال: اگر نمره اخذ شده از سوال ۱ برابر ۱۰۵ و نمره سوال

۲ برابر ۹۵ باشد، نمره نهایی تمرین ۹۷.۵ خواهد بود و نه ۱۰۰.

- لطفا گزارش، کدها و سایر ضمایم را به در یک پوشه با نام زیر قرار داده و آن را فشرده سازید، سپس در سامانه‌ی Elearn بارگذاری نمایید:

HW[Number]_[Lastname]_[StudentNumber]_[Lastname]_[StudentNumber].zip

(مثال: HW1_Ahmadi_810199101_Bagheri_810199102.zip)

- برای گروه‌های دو نفره، بارگذاری تمرین از جانب یکی از اعضا کافی است ولی پیشنهاد می‌شود هر دو نفر بارگذاری نمایند.

پرسش 1. تحلیل عملکرد شبکه های عصبی تحت حملات متخاصم

در این سؤال ما چند شبکه ی CNN و ViT را آموزش داده و عملکرد آن ها را بر روی داده های نویزی و تحت حملات آزمایش می کنیم، با استفاده از آموزش Adversarial در برابر حملات دفاع خواهیم کرد و به چند سؤال پاسخ می دهیم.

1-1. آموزش ResNet روی تصاویر نویزی

در این بخش از مدل ResNet (ResNet34) استفاده کرده و آن را روی مجموعه داده CIFAR-100 با نویز و بدون نویز آموزش می دهیم تا تأثیر نویز بر عملکرد مدل را بررسی کنیم.

1-1-1. بارگذاری مجموعه داده

در گام اول با استفاده از PyTorch مجموعه داده را بارگذاری، نرمال سازی (برای استفاده در شبکه) می کنیم و DataLoader های مناسب را تعریف می کنیم.

1-1-2. افزودن نویز

برای افزودن نویز، کلاسی به نام AddGaussianNoise تعریف می کنیم که در مرحله ی تبدیل (هنگام بارگذاری داده) مورد استفاده قرار گرفته و نویز را به داده ها اضافه می کند (قبل از نرمال سازی). برای اطمینان از صحت فرایند، یک تصویر تصادفی و نسخه ی نویزی آن را نمایش می دهیم.

Original Image

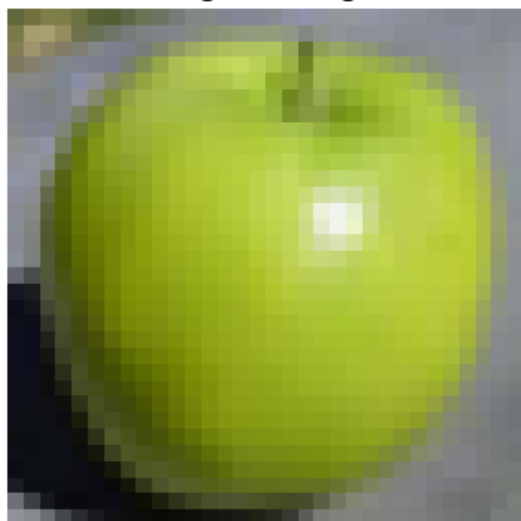


Image with Gaussian Noise



شکل 1: نمونه یک عکس بدون نویز و یک عکس با نویز (عکس ها از دو کلاس متفاوت هستند، چرا که بعد از درست کردن data loader ها تصاویر بر (shuffle) میخورند به همین جهت عکس نویز دار حاصل اضافه کردن نویز به عکس نمایش داده شده نیست!)

3-1-1. آموزش مدل ResNet

در این بخش با استفاده از ResNet34 موجود در کتابخانه‌ی PyTorch (از torchvision.models) و تعریف تابعی برای حلقه‌ی آموزش مدل به مدت ۲۰ اپوک و ذخیره‌ی نتایج برای رسم در بخش بعدی، مدل را آموزش می‌دهیم. این فرایند را یک بار با داده‌های عادی و یک بار با داده‌های نویزی انجام داده و نتایج را مقایسه می‌کنیم. همچنین روش‌های مختلف تنظیم ابرپارامتر مانند منظم‌سازی L2، افزودن لایه Dropout، Learning rate scheduler و آگمنتیشن را نیز امتحان خواهیم کرد.

```
Epoch 15/20 | Train Loss: 0.2768 | Train Acc: 91.13% | Val Loss: 3.4610 | Val Acc: 42.03%
Epoch 16/20 | Train Loss: 0.2528 | Train Acc: 91.92% | Val Loss: 3.4972 | Val Acc: 42.98%
Epoch 17/20 | Train Loss: 0.2267 | Train Acc: 92.73% | Val Loss: 3.7215 | Val Acc: 42.65%
Epoch 18/20 | Train Loss: 0.2078 | Train Acc: 93.33% | Val Loss: 3.7013 | Val Acc: 42.67%
Epoch 19/20 | Train Loss: 0.1910 | Train Acc: 93.80% | Val Loss: 3.7789 | Val Acc: 44.00%
Epoch 20/20 | Train Loss: 0.1814 | Train Acc: 94.07% | Val Loss: 3.7972 | Val Acc: 42.52%
Finished Training. Total time: 5.28 minutes

Training the model on noisy data
Epoch 1/20 | Train Loss: 3.7078 | Train Acc: 13.58% | Val Loss: 3.5892 | Val Acc: 16.74%
...
Epoch 18/20 | Train Loss: 0.3484 | Train Acc: 89.02% | Val Loss: 3.5592 | Val Acc: 41.67%
Epoch 19/20 | Train Loss: 0.2985 | Train Acc: 90.57% | Val Loss: 3.8858 | Val Acc: 40.86%
Epoch 20/20 | Train Loss: 0.2693 | Train Acc: 91.57% | Val Loss: 3.8394 | Val Acc: 40.27%
Finished Training. Total time: 5.26 minutes
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

شکل 2: نتیجه آموزش مدل‌ها بدون L2, Dropout و ...

همان‌طور که مشاهده می‌کنیم، مدل‌ها به‌وضوح دچار بیش‌برازش (Overfitting) شده‌اند.

برای بررسی اینکه آیا می‌توان عملکرد مدل‌ها را بهبود بخشید یا نه، در مرحله بعد به مدل‌های L2 Regularization اضافه خواهیم کرد.

```
Epoch 15/20 | Train Loss: 0.3720 | Train Acc: 88.16% | Val Loss: 2.9968 | Val Acc: 43.55%
Epoch 16/20 | Train Loss: 0.3260 | Train Acc: 89.57% | Val Loss: 3.1686 | Val Acc: 42.41%
Epoch 17/20 | Train Loss: 0.2912 | Train Acc: 90.87% | Val Loss: 3.3442 | Val Acc: 41.98%
Epoch 18/20 | Train Loss: 0.2722 | Train Acc: 91.14% | Val Loss: 3.2315 | Val Acc: 43.15%
Epoch 19/20 | Train Loss: 0.2326 | Train Acc: 92.59% | Val Loss: 3.3310 | Val Acc: 43.43%
Epoch 20/20 | Train Loss: 0.2601 | Train Acc: 91.71% | Val Loss: 3.4063 | Val Acc: 42.38%
Finished Training. Total time: 5.35 minutes

Training the model on noisy data
Epoch 1/20 | Train Loss: 3.6993 | Train Acc: 13.64% | Val Loss: 3.3879 | Val Acc: 19.68%
...
Epoch 18/20 | Train Loss: 0.4177 | Train Acc: 86.87% | Val Loss: 3.1131 | Val Acc: 42.21%
Epoch 19/20 | Train Loss: 0.3762 | Train Acc: 88.00% | Val Loss: 3.0820 | Val Acc: 44.03%
Epoch 20/20 | Train Loss: 0.3642 | Train Acc: 88.38% | Val Loss: 3.4445 | Val Acc: 40.64%
Finished Training. Total time: 5.35 minutes
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

شکل 3: نتیجه مدل‌ها بعد از اضافه کردن L2 Regularization

حتی پس از افزودن L2 Regularization، هر دو مدل همچنان دچار بیش‌برازش)

Overfitting) شدند، با این تفاوت که دقت نهایی آن‌ها روی داده‌های آموزشی کاهش یافت. در مرحله بعد، یک لایه Dropout قبل از لایه نهایی طبقه‌بندی اضافه خواهیم کرد.

```
Epoch 14/20 | Train Loss: 0.9840 | Train Acc: 70.59% | Val Loss: 2.5504 | Val Acc: 43.92%
Epoch 15/20 | Train Loss: 0.8572 | Train Acc: 73.87% | Val Loss: 2.5730 | Val Acc: 43.45%
Epoch 16/20 | Train Loss: 0.7517 | Train Acc: 76.79% | Val Loss: 2.7160 | Val Acc: 43.96%
Epoch 17/20 | Train Loss: 0.6596 | Train Acc: 79.45% | Val Loss: 2.8055 | Val Acc: 43.47%
Epoch 18/20 | Train Loss: 0.5747 | Train Acc: 81.87% | Val Loss: 2.8561 | Val Acc: 44.20%
Epoch 19/20 | Train Loss: 0.5138 | Train Acc: 83.66% | Val Loss: 2.9138 | Val Acc: 44.32%
Epoch 20/20 | Train Loss: 0.4676 | Train Acc: 85.11% | Val Loss: 2.9766 | Val Acc: 43.55%
Finished Training. Total time: 5.34 minutes

Training the model on noisy data
Epoch 1/20 | Train Loss: 4.0059 | Train Acc: 9.59% | Val Loss: 3.5574 | Val Acc: 15.59%
...
Epoch 18/20 | Train Loss: 0.8299 | Train Acc: 74.67% | Val Loss: 2.8788 | Val Acc: 41.97%
Epoch 19/20 | Train Loss: 0.7579 | Train Acc: 76.65% | Val Loss: 2.8271 | Val Acc: 43.07%
Epoch 20/20 | Train Loss: 0.6914 | Train Acc: 78.76% | Val Loss: 2.9047 | Val Acc: 41.86%
Finished Training. Total time: 5.35 minutes
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Figure 1: نتیجه مدل‌ها بعد از اضافه کردن Dropout

Dropout نیز تأثیر قابل‌توجهی نداشت، بنابراین در مرحله بعد از Scheduler استفاده می‌کنیم.

```
Epoch 16/20 | Train Loss: 0.7185 | Train Acc: 77.80% | Val Loss: 2.7425 | Val Acc: 42.91%
Epoch 17/20 | Train Loss: 0.6312 | Train Acc: 80.34% | Val Loss: 2.8110 | Val Acc: 43.56%
Epoch 18/20 | Train Loss: 0.5668 | Train Acc: 82.22% | Val Loss: 2.8547 | Val Acc: 44.00%
Epoch 19/20 | Train Loss: 0.5124 | Train Acc: 83.67% | Val Loss: 3.0034 | Val Acc: 43.37%
Epoch 20/20 | Train Loss: 0.4472 | Train Acc: 85.74% | Val Loss: 3.0352 | Val Acc: 43.67%
Finished Training. Total time: 5.34 minutes

Training the model on noisy data
Epoch 1/20 | Train Loss: 4.0376 | Train Acc: 9.25% | Val Loss: 3.6406 | Val Acc: 14.57%
Epoch 2/20 | Train Loss: 3.4430 | Train Acc: 17.32% | Val Loss: 3.3259 | Val Acc: 20.71%
Epoch 3/20 | Train Loss: 3.1082 | Train Acc: 23.29% | Val Loss: 2.9184 | Val Acc: 26.28%
...
Epoch 18/20 | Train Loss: 0.8682 | Train Acc: 73.53% | Val Loss: 2.6329 | Val Acc: 43.89%
Epoch 19/20 | Train Loss: 0.7906 | Train Acc: 75.96% | Val Loss: 2.8124 | Val Acc: 42.56%
Epoch 20/20 | Train Loss: 0.7190 | Train Acc: 77.89% | Val Loss: 2.8407 | Val Acc: 42.15%
Finished Training. Total time: 5.36 minutes
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

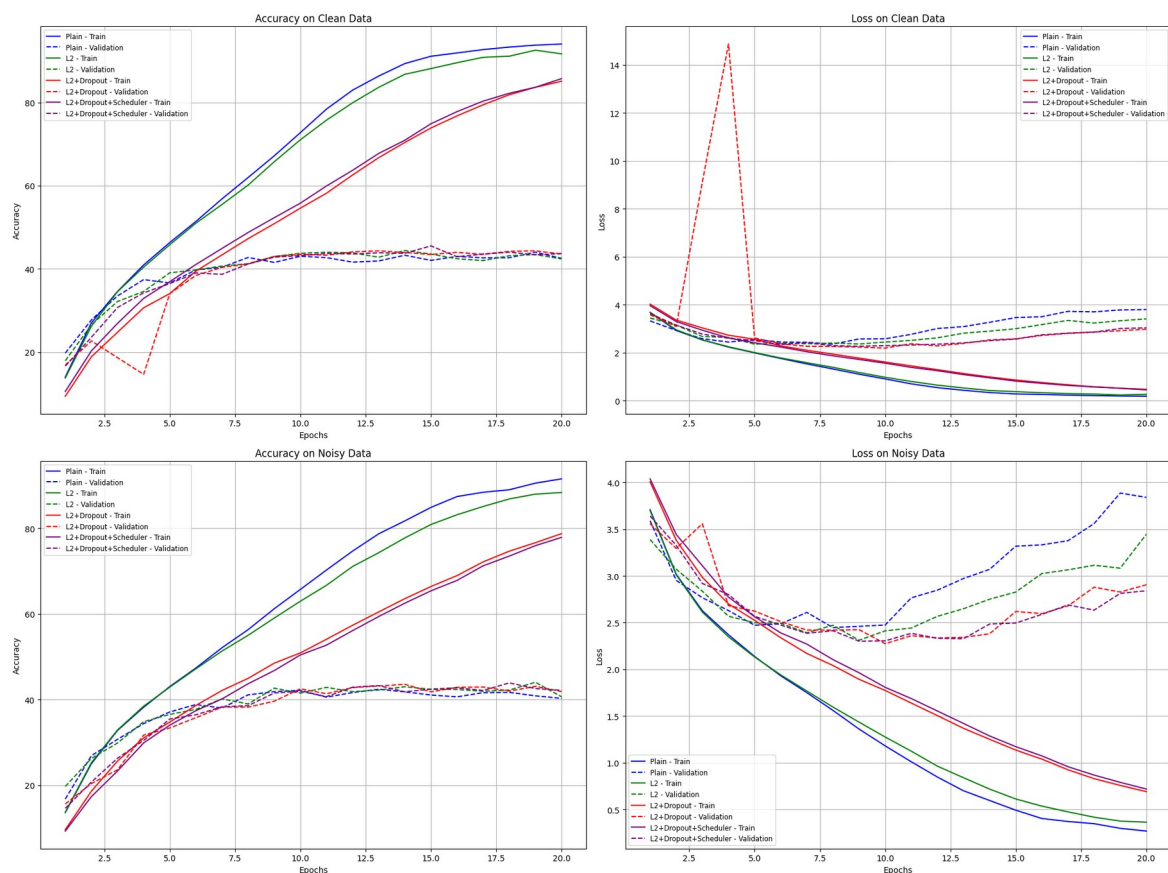
شکل 4: نتیجه مدل‌ها بعد از اضافه کردن Scheduler

۴-۱-۱. نمودارهای دقت و خطا

پس از آموزش مدل‌ها، برای درک بهتر عملکرد آن‌ها، دقت و خطا را برای داده‌های آموزش و آزمایش (Validation) تمام مدل‌هایی که تاکنون آموزش داده‌ایم، ترسیم می‌کنیم.

چهار نمودار رسم می‌کنیم که دقت و خطا برای داده‌های معمولی و داده‌های نویزی، و در هر چهار نمودار، تمام مدل‌ها را نمایش خواهیم داد.

Model Performance Comparison with Different Regularization Techniques



شکل 5: نمودار عملکرد مدل های ResNet

همان طور که مشاهده می کنیم، تمام مدل ها عملکرد نسبتاً مشابهی روی داده های اعتبارسنجی (که مهم تر از آموزش است) داشته اند. بنابراین، نمی توان صرفاً بر اساس این نمودارها درباره عملکرد مدل ها قضاوت کرد و در بخش بعدی به کمک جدول نتیجه گیری نهایی را انجام می دهیم. نکته ای که در این نمودارها جلب توجه می کند، این است که همه مدل ها تقریباً از حدود epoch 7 به بعد دچار بیش برآزش شده اند؛ به طوری که خطا روی داده های اعتبارسنجی شروع به افزایش کرده و دقت نیز بدون تغییر باقی مانده است.

۱-۱-۵. مقایسه عملکرد مدل ها

برای تحلیل بهتر عملکرد مدل ها، بهترین نتایج هر مدل را استخراج کرده و در قالب یک جدول ارائه می دهیم.

Training Data		Strategy	Best Val Accuracy (%)	Best Val Loss
Clean	1. Plain		44.00	2.3560
Clean	2. L2		44.40	2.3540
Clean	3. L2 + Dropout		44.32	2.1903
Clean	4. L2 + Dropout + Scheduler		45.51	2.2650
Noisy	1. Plain		42.41	2.4462
Noisy	2. L2		44.03	2.3112
Noisy	3. L2 + Dropout		43.55	2.2740
Noisy	4. L2 + Dropout + Scheduler		43.89	2.3010

شکل 6: جدول عملکرد مدل های ResNet

از این جدول می‌توان دید که بهترین مدل روی داده‌های تمیز، مدلی بوده که از تمام روش‌ها به صورت همزمان استفاده کرده است. بهترین عملکرد روی داده‌های نویزی نیز مربوط به مدلی است که فقط از L2 استفاده کرده بود. اما از آنجایی که عملکرد مدل L2 + Dropout + Scheduler نیز بسیار نزدیک به مدل L2 بود، در ادامه پروژه مدلی را انتخاب می‌کنیم که از همه روش‌ها استفاده کرده است (این مدل را مجدداً آموزش خواهیم داد و بهترین وزن‌ها را در طول فرآیند آموزش ذخیره خواهیم کرد).

اگر نتایج روی داده‌های تمیز (۴۵٫۵۱٪) و نویزی (۴۳٫۸۹٪) را با یکدیگر مقایسه کنیم، متوجه می‌شویم که با وجود اختلال نویز در داده‌ها، مدل توانسته تا حدودی نویز را فیلتر کند و کاهش عملکرد کمتر از ۲٪ بوده است. این موضوع ممکن است به دلیل اندازه نسبتاً بزرگ شبکه ResNet و قابلیت تعمیم‌دهی خوب آن باشد که به آن اجازه می‌دهد نویز را به راحتی فیلتر کند.

در مورد عملکرد کلی پایین مدل نیز می‌توان دو فرضیه مطرح کرد:

۱. داده‌های ورودی به اندازه کافی زیاد نبوده‌اند.
۲. مدل به اندازه‌ای پیچیده نبوده که بتواند داده‌ها را به خوبی تفکیک کند.

با توجه به اینکه ResNet یک مدل نسبتاً بزرگ و قدرتمند است، فرضیه اول محتمل‌تر است و برای بهبود عملکرد می‌توان از افزایش داده (Data Augmentation) استفاده کرد؛ اما چون هدف این پروژه بررسی تأثیر نویز بوده، از انجام آن صرف‌نظر می‌کنیم.

۱-۲. یادگیری انتقالی روی مجموعه داده Flowers-102 با ViT

۱-۲-۰. بارگذاری و پیش‌پردازش مجموعه داده

در این بخش از مجموعه داده Flowers-102 استفاده خواهیم کرد که شامل تصاویر گل‌های رایج در بریتانیا است و شامل ۱۰۲ کلاس مختلف با بیش از ۸۰۰۰ تصویر می‌باشد.

در این مرحله، مجموعه داده را دانلود کرده و لودرهای مناسب داده را آماده خواهیم کرد. همچنین تصاویر را به گونه‌ای Transform می‌کنیم که با مدل ViT سازگار باشند. برای بهبود عملکرد مدل، از چرخش‌های تصادفی و Random Horizontal Flip نیز برای افزایش داده (Data Augmentation) استفاده خواهیم کرد.



شکل 7: نمونه تصاویر Flowers-102 بعد از لود

۱-۲-۱. بارگذاری مدل از پیش آموزش‌دیده

در این بخش، مدل از پیش آموزش‌دیده vit_base_patch16_224 را از کتابخانه torchvision بارگذاری کردیم و لایه خروجی آن را تغییر خواهیم داد تا با ۱۰۲ کلاس موجود در مجموعه داده ما سازگار شود.

از آنجا که قصد داریم از یادگیری انتقالی (Transfer Learning) استفاده کنیم، تمام پارامترهای بخش پشتی (Backbone) مدل را فریز می‌کنیم و تنها اجازه می‌دهیم که لایه نهایی طبقه‌بندی آموزش ببیند.

```
Linear(in_features=768, out_features=102, bias=True)
Total parameters: 85,877,094
Trainable parameters: 78,438
```

شکل 8: تعداد پارامترهای مدل از پیش آموزش دیده ViT

۱-۲-۲ Fine Tuning روی Flowers-102

در این مرحله، مدل را به مدت ۵ epoch روی مجموعه داده Flowers-102 Fine tune خواهیم کرد و بهترین مدل را ذخیره می‌کنیم.

برای دستیابی به بهترین نتایج، پس از Fine tune، به نرخ یادگیری (learning rate) برابر با ۰.۰۰۱ و ضریب تنظیم (weight_decay) برابر با ۰.۰۰۰۱ رسیدیم.

```
Fine-tuning the ViT model on Flowers-102
Epoch 1: New best model saved with Val Acc: 54.41%
Epoch 1/5 | Train Loss: 3.9354 | Train Acc: 22.55% | Val Loss: 2.7482 | Val Acc: 54.41%
Epoch 2: New best model saved with Val Acc: 69.90%
Epoch 2/5 | Train Loss: 1.9592 | Train Acc: 71.57% | Val Loss: 1.8346 | Val Acc: 69.90%
Epoch 3: New best model saved with Val Acc: 75.39%
Epoch 3/5 | Train Loss: 1.1454 | Train Acc: 87.55% | Val Loss: 1.4131 | Val Acc: 75.39%
Epoch 4: New best model saved with Val Acc: 77.75%
Epoch 4/5 | Train Loss: 0.7369 | Train Acc: 93.24% | Val Loss: 1.1850 | Val Acc: 77.75%
Epoch 5: New best model saved with Val Acc: 81.18%
Epoch 5/5 | Train Loss: 0.5134 | Train Acc: 95.98% | Val Loss: 1.0432 | Val Acc: 81.18%
Finished Training. Total time: 3.06 minutes
<All keys matched successfully>
```

شکل 9: روند آموزش مدل از پیش آموزش دیده ViT

مدل از پیش آموزش دیده توانست به دقت قابل قبول ۸۱٪ روی داده‌های اعتبارسنجی دست یابد. با نگاهی به روند آموزش، علائمی از بیش‌برازش (Overfitting) مشاهده می‌شود، بنابراین می‌توان گفت حتی با افزایش تعداد Epoch ها، احتمالاً عملکرد مدل بهبود چندانی نخواهد یافت.

۱-۲-۲ بارگذاری و آموزش ViT از ابتدا

در این بخش، همان مدل قبلی را بدون استفاده از وزن‌های پیش‌آموزش‌دیده بارگذاری کرده و آن را از ابتدا روی دیتاست آموزش می‌دهیم. در صورت سؤال ذکر شده بود که مدل برای حدود ۱۰ اپک آموزش

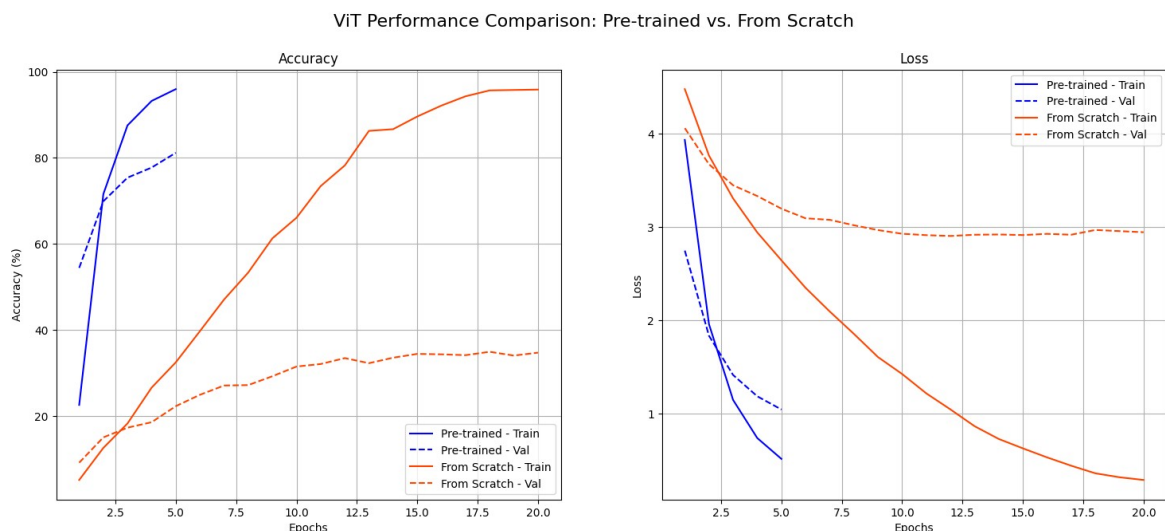
داده شود، اما چون هنوز جای بهبود وجود داشت، آموزش را تا ۲۰ اپک ادامه دادیم.

پس از Fine tune هایپرپارامترها، بهترین عملکرد مدل با استفاده از منظم‌ساز L2 با مقدار $1e-4$ و نرخ یادگیری $1e-5$ به دست آمد.

```
Epoch 9: New best model saved with Val Acc: 30.10%
Epoch 9/20 | Train Loss: 1.6970 | Train Acc: 57.65% | Val Loss: 2.9977 | Val Acc: 30.10%
Epoch 10/20 | Train Loss: 1.4619 | Train Acc: 64.90% | Val Loss: 3.0409 | Val Acc: 29.02%
Epoch 11/20 | Train Loss: 1.2861 | Train Acc: 70.88% | Val Loss: 3.0065 | Val Acc: 29.31%
Epoch 12: New best model saved with Val Acc: 32.16%
Epoch 12/20 | Train Loss: 1.1032 | Train Acc: 76.57% | Val Loss: 2.9495 | Val Acc: 32.16%
Epoch 13/20 | Train Loss: 0.9092 | Train Acc: 82.75% | Val Loss: 2.9525 | Val Acc: 31.86%
Epoch 14: New best model saved with Val Acc: 32.65%
...
Epoch 19: New best model saved with Val Acc: 35.29%
Epoch 19/20 | Train Loss: 0.3345 | Train Acc: 95.00% | Val Loss: 2.9652 | Val Acc: 35.29%
Epoch 20/20 | Train Loss: 0.2994 | Train Acc: 95.49% | Val Loss: 2.9707 | Val Acc: 34.80%
Finished Training. Total time: 11.60 minutes
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
<All keys matched successfully>
```

شکل 10: روند یادگیری مدل بدون آموزش اولیه

مدل پس از ۲۰ اپک آموزش و ذخیره بهترین نسخه، موفق شد به دقت ۳۵٪ روی داده‌های اعتبارسنجی برسد، اما علائم واضحی از بیش‌برازش (overfitting) مشاهده می‌شود که با توجه به اندازه بزرگ مدل ViT و کوچک بودن دیتاست ما قابل انتظار بود.



شکل 11: نمودار خطا و دقت مدل های ViT در طول آموزش

می بینیم که هر دو مدل آموزش دیده اند و نشانه هایی از بیش برآزش (خصوصاً در نسخه آموزش داده شده از ابتدا) دیده می شود.

در نهایت، مدل از پیش آموزش دیده به دقت ۸۱٪ و مدل آموزش دیده از ابتدا به دقت ۳۵٪ رسید. این اختلاف چشمگیر در عملکرد، قدرت Pre-training را، به ویژه در مدل های ViT که از ترنسفورمرها استفاده میکنند به خوبی نشان می دهد.

مدل از پیش آموزش دیده روی دیتاست ImageNet آموزش دیده بود که مجموعه ای بسیار بزرگ با بیش از یک میلیون تصویر و ۱۰۰۰ کلاس مختلف است. آموزش روی این دیتاست باعث شده که مدل توانایی استخراج ویژگی هایی غنی از تصاویر را یاد بگیرد. بنابراین وقتی این مدل را روی دیتاست گل ها اعمال کردیم، فقط لازم بود بخش طبقه بند (classifier) آن را آموزش دهیم تا یاد بگیرد چگونه از ویژگی هایی که قبلاً یاد گرفته برای تشخیص گل ها استفاده کند.

در مقابل، مدل آموزش داده شده از ابتدا فقط به دیتاست محدود گل ها دسترسی داشت. برخلاف مدل های CNN که تا حدی دانش ذاتی نسبت به ساختار تصویر (لبه ها، شکل ها و...) دارند، مدل های Vision Transformer چنین دانشی ندارند و باید این مفاهیم پایه را از صفر یاد بگیرند. این موضوع، همراه با تعداد زیاد پارامترهای این مدل ها، باعث می شود که آن ها برای عملکرد خوب به داده های بسیار بیشتری نیاز داشته باشند؛ چیزی که در پروژه ی ما فراهم نبود. بنابراین، همان طور که مشاهده شد، مدل به احتمال زیاد دچار بیش برآزش شده و عملکرد پایینی داشته است.

۱-۳. حملات متخاصم و دفاع

در این بخش قصد داریم عملکرد مدل‌های قبلی را در برابر حملات FGSM و PGD بررسی کنیم و آن‌ها را با استفاده از Adversarial Training مقاوم سازی کنیم!

قبل از شروع، تعاریف زیر را از یکی از گزارش‌های قبلی‌ام کپی کرده‌ام تا این مفاهیم با جزئیات بیشتری توضیح داده شوند:

حملات خصمانه (Adversarial attacks):

طبق توضیح مقاله، حمله خصمانه به تصویر اصلی یک آشفتگی بسیار کوچک و غیرقابل تشخیص اضافه می‌کند تا تصویری جدید بسازد که هدف آن بیشینه‌سازی خطا یا Loss مدل است.

Fast Gradient Sign Method (FGSM):

براساس این توضیح در TensorFlow:

https://www.tensorflow.org/tutorials/generative/adversarial_fgsm

FGSM یک روش سریع برای تولید مثال‌های خصمانه است. این روش با بهره‌گیری از نحوه رفتار تابع Loss مدل، با تنها یک تغییر در هر پیکسل تصویر ورودی، Loss مدل را بیشینه می‌کند. مراحل کار این روش به صورت زیر است:

1. محاسبه گرادیان: ابتدا گرادیان تابع Loss مدل را نسبت به هر پیکسل تصویر ورودی محاسبه می‌کند تا جهت افزایش بیشترین Loss مشخص شود.

2. به دست آوردن علامت: به جای استفاده مستقیم از مقدار گرادیان، فقط علامت هر عنصر گرادیان استفاده می‌شود؛ یعنی برای هر پیکسل مشخص می‌کند که مقدار آن باید افزایش (+1) یا کاهش (-1) یابد.

3. اعمال آشفتگی: بردار علامت در یک عدد اسکالر کوچک (اپسلون) ضرب می‌شود تا بزرگی کلی آشفتگی کنترل شود.

فرمول محاسبه FGSM:

$$adv_x = x + \epsilon * \text{sign}(\nabla_x J(\theta, x, y))$$

شکل 12: فرمول محاسبه FGSM

Projected Gradient Descent (PGD)

Medium: این توضیح در

<https://medium.com/@zachariahharungeorge/unveiling-the-power-of-projected-gradient-descent-in-adversarial-attacks-2f92509dde3c>

PGD توسعه ایده اصلی حمله خصمانه است. به این صورت که به جای یک گام بزرگ، PGD با چندین گام کوچکتر عمل می‌کند تا شانس یافتن یک آشفستگی موفق را افزایش دهد. مراحل کار:

1. مقداردهی اولیه: با یک گام کوچک و تصادفی از تصویر اصلی دور می‌شود، اما همچنان در محدوده مجاز باقی می‌ماند.
 2. حلقه: به تعداد از پیش تعیین شده ای، گرادیان Loss نسبت به پیکسل‌های تصویر فعلی محاسبه می‌شود و سپس یک گام کوچک به سمت علامت گرادیان برداشته می‌شود. اندازه این گام با پارامتر α مشخص می‌شود (مثلاً $\alpha = 1/255$ در مقاله).
 3. پروجکشن: پس از هر گام، اگر تصویر جدید از محدوده ϵ پیرامون تصویر اصلی خارج شده باشد، با یک عملیات Clipping آشفستگی به گونه‌ای محدود می‌شود که مقدار آن از ϵ تجاوز نکند.
- از آنجا که PGD یک فرایند تکراری (iterative) است و منطقه پیرامون تصویر را با دقت بیشتری جستجو می‌کند، حمله‌ای قوی‌تر از FGSM محسوب می‌شود و یکی از معیارهای استاندارد برای ارزیابی مقاومت مدل در برابر حملات خصمانه است.

۱-۳-۱. پیاده‌سازی FGSM و PGD

در این بخش، چند تابع پیاده‌سازی خواهیم کرد که یک مدل، یک دسته از تصاویر و برچسب‌های آن‌ها، به همراه پارامترهای حمله را دریافت کرده و یک دسته تصویر متناظر برای حمله تولید می‌کنند.

FGSM ۱-۳-۱-۱

حمله‌ی FGSM با برداشتن یک گام در جهت گرادیان تابع زیان نسبت به تصویر ورودی عمل می‌کند. بنابراین دقیقاً همین کار را انجام می‌دهیم و در انتها تصویر را به محدوده مجاز مقدار پیکسل‌ها محدود می‌کنیم تا از بروز خطا جلوگیری شود.

```

1 def fgsm_attack(model, loss_function, image, label, epsilon):
2     image.requires_grad = True
3
4     output = model(image)
5
6     loss = loss_function(output, label)
7
8     model.zero_grad()
9
10    loss.backward()
11
12    gradient = image.grad.data
13
14    sign_gradient = gradient.sign()
15
16    perturbed_image = image + epsilon * sign_gradient
17
18    perturbed_image = torch.clamp(perturbed_image, image.min(), image.max())
19
20    return perturbed_image

```

شکل 13: تابع fgsm

PGD ۱-۳-۱-۲

PGD یا Projected Gradient Descent نسخه‌ای قدرتمندتر از FGSM است. به جای اینکه فقط یک گام بزرگ بردارد، چندین گام کوچک برمی‌دارد و پس از هر گام، نتیجه را به ناحیه مجاز (توپ e) اطراف تصویر اصلی بازمی‌گرداند. بنابراین برای پیاده‌سازی آن باید این فرآیند را چندین بار تکرار کنیم.

```

1 def pgd_attack(model, loss_function, image, label, epsilon, alpha, iterations):
2     original_image = image.data.clone().detach()
3
4     perturbed_image = image.data.clone().detach()
5
6     for i in range(iterations):
7         perturbed_image.requires_grad = True
8
9         output = model(perturbed_image)
10
11         loss = loss_function(output, label)
12
13         model.zero_grad()
14
15         loss.backward()
16
17         gradient_sign = perturbed_image.grad.data.sign()
18
19         perturbed_image = perturbed_image.detach() + alpha * gradient_sign
20
21         eta = torch.clamp(perturbed_image - original_image, min=-epsilon, max=epsilon)
22
23         perturbed_image = torch.clamp(original_image + eta, min=image.min(), max=image.max())
24
25     return perturbed_image

```

شکل 14: تابع pgd

۲-۳-۱. تولید تصاویر متخاصم و ارزیابی مدل‌ها

سپس در این بخش تصاویر متخاصم را برای مدل‌های انتخابی مان (ResNet آموزش‌دیده روی داده‌های پاک، ResNet آموزش‌دیده روی داده‌های نویزی، ViT از پیش آموزش‌دیده و ViT آموزش‌دیده از اول) تولید کرده و عملکرد آن‌ها را روی این نمونه‌های مقابله‌ای ارزیابی کنیم.

Model	Natural Acc. (%)	FGSM Acc. (%)	PGD Acc. (%)
ResNet (Clean)	46.560000	3.370000	0.990000
ResNet (Noisy)	45.230000	4.750000	1.630000
ViT (Pre-trained)	78.370467	1.040820	0.000000
ViT (From Scratch)	30.248821	0.146365	0.032526

شکل 15: نتایج مدل‌های بعد از حمله

* جور در نیامدن دقت مدل‌ها با بخش‌های قبل به این دلیل است که مجبور شدم چندین بار کد را اجرا کنم! :

همان‌طور که مشاهده می‌کنید، هر دو حمله تأثیر قابل توجهی بر روی مدل‌ها داشتند؛ حمله FGSM توانست عملکرد مدل‌های ResNet را از حدود ۴۵٪ به کمتر از ۵٪ کاهش دهد و عملکرد ViT را به ۱٪ برساند!

اما همان‌طور که انتظار می‌رفت، PGD (که حمله‌ای پیچیده‌تر است) عملکرد ResNet‌ها را به کمتر از ۱.۷٪ و عملکرد ViT را به ۰٪ کاهش داد! نکته جالب این است که مدل ResNet که روی داده‌های نویزی آموزش دیده بود، کمی مقاوم‌تر در برابر این حملات ظاهر شد؛ که نشان می‌دهد افزودن نویز می‌تواند در افزایش مقاومت مدل در برابر حملات مقابله‌ای مؤثر باشد!

۱-۳-۳ Adversarial Training

در این بخش قصد داریم Adversarial training را روی هر چهار مدل انجام دهیم تا آن‌ها را در برابر حملات مقاوم‌تر کنیم. برای این کار، یک تابع آموزش جدید تعریف می‌کنیم که مشابه تابع آموزش معمولی است، با این تفاوت جزئی که به جای تصاویر معمولی، از تصاویر متخاصم استفاده می‌کند. برای حفظ پایداری در روند آموزش، از حمله‌ای کمی ضعیف‌تر استفاده خواهیم کرد.

یک نکته مهم این است که به دلیل محدودیت حافظه گرافیکی (VRAM)، مجبور شدم کد را چندین بار اجرا کرده و نتایج نهایی را به صورت دستی یادداشت کنم، و در طول اجرا‌های مختلف، عملکرد مدل‌ها کمی متفاوت بود.

Model	Clean Acc	FGSM Acc (Before)	PGD Acc (Before)	Clean Acc (After)	FGSM Acc (After)	PGD Acc (After)
ResNet (Clean Data)	47.52	3.39	0.94	42.78	14.97	12.31
ResNet (Noisy Data)	44.27	4.49	1.43	40.17	14.03	11.83
ViT (Pre-trained)	79.70	0.96	0.00	60.99	21.34	15.71
ViT (From Scratch)	30.92	0.05	0.00	19.21	3.63	3.22

شکل 16: عملکرد مدل‌ها بعد از Adversarial training

با توجه به جدول می‌توان مشاهده کرد که به‌طور کلی دقت مدل‌های ResNet پس از Adversarial training (روی داده‌های پاک) افت زیادی نداشته و در مقابل، مقاومت آن‌ها در برابر حملات به‌طور قابل توجهی افزایش یافته است (از ۳.۴٪ و ۱٪ به ۱۵٪ و ۱۲.۳٪ رسید).

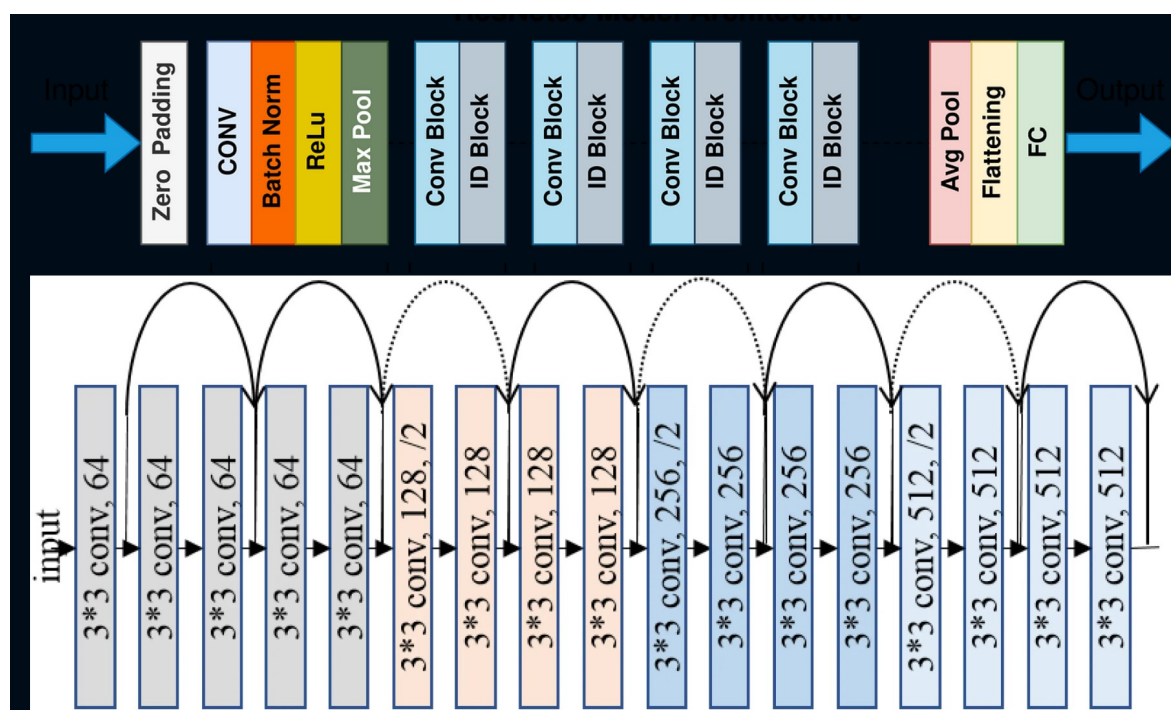
اما مدل‌های ViT افت عملکرد بسیار بیشتری روی داده‌های پاک پس از Adversarial training تجربه کردند (کاهش ۲۰٪ برای نسخه از پیش آموزش‌دیده و ۱۰٪ برای نسخه دیگر). نسخه از پیش آموزش‌دیده‌ی ViT بهبود قابل توجهی

روی تصاویر خصمانه نشان داد (از ۰٪ به ۲۱٪ و ۱۶٪ رسید) که آن را به بهترین مدل از نظر عملکرد در برابر حملات تبدیل کرد. اما نسخه‌ی دیگر ViT تنها ۳٪ بهبود را تجربه کرد.

1-4. سؤالات تئوری

1-4-1. با وجود اینکه مدل ResNet دارای میلیون‌ها پارامتر است، چرا در بسیاری از موارد با داده‌های نویزی همچنان عملکرد مناسبی دارد؟

مدل‌هایی با تعداد پارامتر بالا بیشتر مستعد بیش‌برازش (overfitting) و حساس‌تر به نویز هستند ولی معماری ResNet آن را در برابر نویز مقاوم می‌سازد. دلیل اصلی این موضوع استفاده از مؤلفه‌های زیر است (که در تصویر زیر نشان داده شده‌اند که ساختار ResNet را نمایش می‌دهد):



شکل 17: ساختار ResNet

Residual Connections

این نوآوری اصلی در ResNet است. با اضافه کردن skip connection ها، که به مدل اجازه می‌دهد Residual Function ها را یاد بگیرد، به طور مؤثر به مدل اجازه می‌دهیم در صورت لزوم نگاشت ساده‌تری (مثل تابع همانی) را انتخاب کند، مثلاً با عبور دادن خروجی بلاک قبلی به بلاک بعدی. این کار بسیار ساده‌تر از این است که چندین لایه غیرخطی را مجبور به تقریب تابع همانی کنیم. همچنین این کار باعث نرم‌تر شدن تابع هزینه می‌شود و تابع هزینه نرم‌تر باعث افزایش پایداری مدل در برابر نویز می‌شود.

Ensemble Behavior

استفاده از بلاک‌های Residual باعث می‌شود ResNet رفتار مشابه با یک مجموعه ensemble از چندین شبکه را داشته باشد. مسیرهای مختلفی که یک سیگنال می‌تواند از طریق شبکه طی کند (یا عبور از یک بلاک یا عبور از مسیر پرشی (skip connection)) مانند مدل‌های مختلف عمل می‌کنند و خروجی نهایی ترکیبی از این مسیرها خواهد بود. همانطور که می‌دانیم، ensemble تکنیکی قدرتمند برای افزایش پایداری مدل است.

Batch Normalization

ResNet از لایه‌های batch normalization زیادی استفاده می‌کند (تقریباً بعد از هر لایه کانولوشن). batch normalization ورودی‌های هر لایه را برای هر مینی‌بچ استاندارد می‌کند که هم باعث افزایش پایداری فرآیند آموزش می‌شود و هم نقش منظم‌کننده (regularizer) را ایفا می‌کند، که باعث پایداری و مقاومت بیشتر در برابر نویز می‌شود.

مجموعه‌ی این عوامل باعث شده‌اند که مدل ResNet نسبت به نویز حساسیت کمی داشته باشد، همان‌طور که در نتایج ما نیز دیده شد.

Training Data		Strategy	Best Val Accuracy (%)	Best Val Loss
Clean	1. Plain		44.00	2.3560
Clean	2. L2		44.40	2.3540
Clean	3. L2 + Dropout		44.32	2.1903
Clean	4. L2 + Dropout + Scheduler		45.51	2.2650
Noisy	1. Plain		42.41	2.4462
Noisy	2. L2		44.03	2.3112
Noisy	3. L2 + Dropout		43.55	2.2740
Noisy	4. L2 + Dropout + Scheduler		43.89	2.3010

شکل 18: نتایج ResNet در برابر نویز

۲-۴-۱. مدل‌های ViT (Transformer Vision) در صورت استفاده از وزن های پیش‌آموزش دیده بهتر از حالت بدون pretrain عمل میکنند. این اختلاف را با استفاده از مفاهیم مینیمم تیز (sharp) و تخت (flat) تحلیل کنید.

ابتدا مفاهیم مینیمم تخت و مینیمم تیز را توضیح میدهیم:

اگر تابع خطا (loss) را مانند یک زمین ناهموار تصور کنیم که نقاط پایین‌تر آن، راه‌حل‌های بهتری هستند (یعنی خطای کمتر دارند)،

مینیمم تخت مانند یک دره پهن و عمیق است که اگر کمی در هر جهت حرکت کنیم، مقدار خطا تغییر زیادی نمی‌کند. اگر مدل بتواند چنین مینیممی را پیدا کند، پایدارتر خواهد بود و قدرت تعمیم بهتری دارد؛ زیرا تغییرات کوچک در ورودی یا وزن‌ها، تأثیر زیادی بر خروجی نمی‌گذارد.

مینیمم تیز در مقابل شبیه یک دره باریک و عمیق است که در پایین‌ترین نقطه‌اش مقدار خطا بسیار کم است، اما کوچک‌ترین حرکت از آن نقطه، باعث افزایش زیاد خطا می‌شود. معمولاً وقتی مدل به یک مینیمم تیز برسد، به این معناست که بیش‌برازش (overfit) رخ داده و فقط داده‌های آموزش را حفظ کرده است.

وقتی که یک مدل ViT را بدون آموزش از قبل روی یک دیتاست کوچک آموزش می‌دهیم، به دلیل اینکه ViT دارای میلیون‌ها پارامتر است و ما داده کمی داریم (در حد ۱۰۰۰)، مدل تمایل به بیش‌برازش دارد و به احتمال زیاد به سمت یک مینیمم تیز می‌رود. در نتیجه، وقتی داده‌های جدیدی از مجموعه اعتبارسنجی به آن داده می‌شود، تفاوت‌های کوچک در تصاویر می‌توانند به راحتی مدل را از این مینیمم تیز خارج کرده و منجر به افزایش خطا و پیش‌بینی اشتباه شوند.

اما وقتی از یک مدل ViT آموزش دیده از قبل استفاده می‌کنیم، مدل از قبل در استخراج ویژگی‌ها خوب عمل می‌کند و فرآیند آموزش قبلی، آن را به ناحیه‌ای از تابع خطا هدایت کرده است که پر از راه‌حل‌های پایدار و قابل اعتماد است. fine-tuning چنین مدلی شبیه جستجو در ناحیه‌ای از تابع خطا است که صاف‌تر است؛ بنابراین، مدل کمتر دچار بیش‌برازش می‌شود و عملکرد بهتری خواهد داشت!

۳-۴-۱. تحلیل آموزش پادماننده (adversarial training) به عنوان شکلی از افزایش داده (data augmentation)

می‌توان آموزش پادماننده را به عنوان شکلی خاص از افزایش داده در نظر گرفت. در اصل، هر دو روش تلاش می‌کنند نمونه‌های جدیدی برای آموزش با اعمال تبدیل‌هایی بر داده‌های موجود ایجاد کنند، با این هدف که مدل نسبت به این تبدیل‌ها ناپایدار نباشد و ویژگی‌های مقاومی یاد بگیرد که به مدل کمک می‌کند بهتر تعمیم دهد. اما تفاوت اصلی آن‌ها در این است که در حالی که افزایش داده از مجموعه‌ای از تبدیل‌های تصادفی از پیش تعریف‌شده (مانند چرخش، برش، قرینه‌سازی، تغییر رنگ و غیره) استفاده می‌کند که هدفشان شبیه‌سازی تغییرات واقع‌گرایانه داده‌هاست، آموزش پادماننده از تبدیل‌های بهینه‌شده‌ای استفاده می‌کند که مخصوص خود مدل هستند تا بدترین سناریوی ممکن را ایجاد کنند، با این هدف که مدل در برابر حملات مقاوم شود. بنابراین می‌توان گفت آموزش پادماننده در اصل شکلی از افزایش داده سخت محسوب می‌شود!

۳-۴-۱. adversarial training را به عنوان شکلی از data augmentation تحلیل کنید.

می‌توان adversarial training را به عنوان شکلی خاص از data augmentation در نظر گرفت. در اصل، هر دو روش تلاش می‌کنند نمونه‌های جدیدی برای آموزش با اعمال تبدیل‌هایی بر داده‌های موجود ایجاد کنند، با این هدف که مدل نسبت به این تبدیل‌ها حساس نباشد و ویژگی‌های مقاومی یاد بگیرد که به مدل کمک می‌کند بهتر تعمیم دهد. اما تفاوت اصلی آن‌ها در این است که در حالی که data augmentation از مجموعه‌ای از تبدیل‌های تصادفی از پیش تعیین شده (مانند چرخش، برش، قرینه‌سازی، تغییر رنگ و ...) استفاده می‌کند که هدفشان شبیه‌سازی تغییرات واقعی داده‌هاست، adversarial training از تبدیل‌های بهینه‌شده‌ای استفاده می‌کند که مخصوص خود مدل هستند تا بدترین سناریوی ممکن را ایجاد کنند، با این هدف که مدل در برابر حملات مقاوم شود. بنابراین می‌توان گفت adversarial training در اصل شکلی از data augmentation سخت محسوب می‌شود!

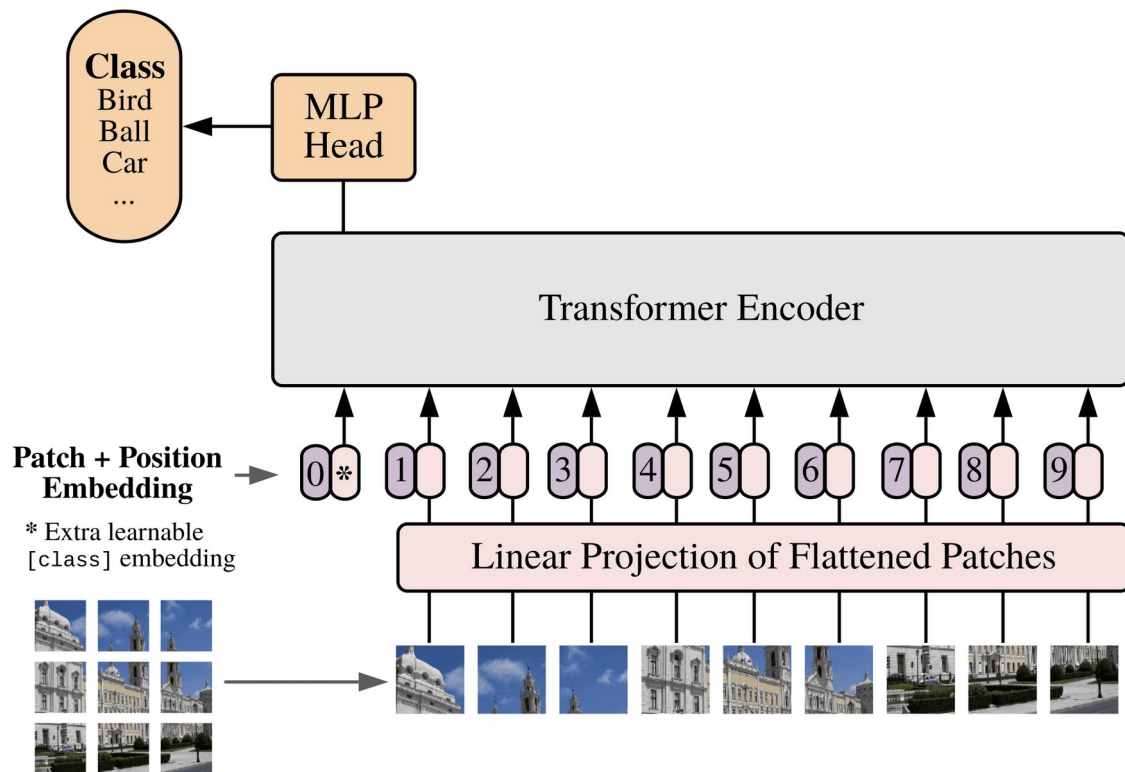
۴-۴-۱. با تحلیل ساختار ResNet و ViT و استفاده از مفاهیم dropout و ensemble ، تفاوت این دو مدل در برابر حملات متخاصم را توجیه کنید .

معماری ResNet به دلایلی که پیش‌تر توضیح داده شد، تا حدی در برابر نویز و حملات مقاوم است، در حالی که مکانیزم self-attention در ViT آن را در برابر حملات آسیب‌پذیرتر می‌کند، اما در عوض ظرفیت بالاتری برای رسیدن به مقاومت پس از adversarial training دارد (به‌طور کلی ظرفیت یادگیری بالاتری دارد).

اگر بخواهیم وارد جزئیات بیشتری شویم، اتصال‌های پرشی (skip connections) در ResNet اجازه می‌دهند سیگنال‌ها از مسیرهای مختلف عبور کنند، که در عمل یک ensemble ضمنی از شبکه‌ها ایجاد می‌کند. همان‌طور که می‌دانیم، ensemble یکی از روش‌های قدرتمند برای افزایش مقاومت مدل است و احتمالاً همین دلیل باعث شده که ResNet در شرایط حمله عملکرد بهتری نسبت به ViT داشته باشد (که در شکل ۱۶ قابل مشاهده است!). علاوه بر این، توانایی پرش از بلوک‌ها به مدل امکان می‌دهد که یاد بگیرد به هیچ بلوکی وابستگی شدید نداشته باشد، و این باعث می‌شود در برابر حملاتی که ممکن است برخی بلوک‌ها را هدف قرار دهند، عملکرد بهتری داشته باشد.

در مقابل، ViT به مکانیزم self-attention متکی است و به‌شکل گسترده‌ای از dropout برای منظم‌سازی استفاده می‌کند (می‌توانید ساختار ViT را در تصویر زیر ببینید). اما drop-out های استاندارد معمولاً در برابر حملات کارایی کمتری دارند، زیرا حمله در زمان inference (زمانی که dropout غیرفعال است) طراحی می‌شود. با این حال، ظرفیت یادگیری بالاتر ViT و استفاده از مکانیزم attention به آن اجازه می‌دهد که پس از adversarial training عملکرد بهتری داشته باشد.

Vision Transformer (ViT)



شكل 19: ساختار ViT